SIMPLIFYING BY IMPLEMENTING DISTRIBUTED SYSTEMS

Distributed Systems Simplified

Consistent Hashing:

Scaling without the rebalancing nightmare.

AUTHOR:

Niv Rave

Github: https://github.com/NivRave/Distributed-Systems-Simplified

The Limits of a Standard Modulo

The Rebalancing Nightmare

In our previous post, we used *Hash(key) % N*. This works - until you need to add a node.

The Problem:
If you move from 3 shards to 4, the value of N changes. This shuffles the destination for almost **every single key**.

The Result:
A "Rehashing Storm." You have to move nearly 100% of your data across the network to new nodes.

The Scaling Wall:

- **Downtime**: Your system crawls while data migrates.
- **Cache Invalidation**: If this were a cache, you'd have a 0% hit rate instantly.

The Fix: Consistent Hashing.

The Architectural Solution

The Ring: Mapping to a Circle

Instead of a linear array, we imagine the hash space as a continuous Ring .

1. **Place Nodes**: Hash your servers (IDs or IPs) onto points on the ring.
2. **Place Keys**: Hash your data onto the same ring.
3. **Assignment**: To find a key's server, simply walk clockwise until you hit the first Node.

The Result: When a node is added, it only "steals" keys from its immediate counter-clockwise neighbor. The rest of the ring stays untouched.

Code: The Storage Node

The Primitive Block

Before building the Ring, we define the individual physical node.

- **Simple Storage**: A thread-safe Map using *sync.RWMutex*.
- **Identity**: Each node has a unique ID (like "Node-A") which determines its fixed position on the Ring.

Below is the implementation under *node.go*:

```go
type Node struct {
    ID   string
    Data map[string]string // The actual Key-Value store
    mu   sync.RWMutex      // Thread-safe access!
}

func (n *Node) Put(key, value string) {
    n.mu.Lock()
    defer n.mu.Unlock()
    n.Data[key] = value
}
```

Code: The Ring Strategy

Coding the Ring Lookup

We use Binary Search to efficiently find the "Next Server" on the ring.

- **Sorted Hashes**: We maintain a sorted slice of all node positions.
- **The Wrap-around**: If a key's hash is higher than the last node, it "wraps around" to the first node in the slice (Index 0).

Below is the implementation under *ring.go*:

```go
func (c *ConsistencyRing) GetNode(key string) string {
    hash := c.hash(key)

    // Binary Search: find first nodeHash >= keyHash
    idx := sort.Search(len(c.sortedHashes), func(i int) bool {
        return c.sortedHashes[i] >= hash
    })

    // Wrap around logic for the circle
    if idx == len(c.sortedHashes) {
        idx = 0
    }
    return c.ring[c.sortedHashes[idx]]
}
```

Code: Solving Hotspots

Virtual Nodes

What if nodes are unevenly spaced? One node might end up responsible for 70% of the ring!

- **Virtual Nodes**: We replicate each physical node multiple times (e.g., "Node-A#1", "Node-A#2").
- **The Benefit**: This scatters the node's influence across the ring, ensuring a near-perfect distribution of data.

Below is the implementation under *ring.go*:

```go
func (c *ConsistencyRing) AddNode(nodeID string) {
    for i := 0; i < c.virtualNodes; i++ {
        virtualKey := fmt.Sprintf("%s#%d", nodeID, i)
        hash := c.hash(virtualKey)

        c.ring[hash] = nodeID
        c.sortedHashes = append(c.sortedHashes, hash)
    }
    sort.Slice(c.sortedHashes, func(i, j int) bool {
        return c.sortedHashes[i] < c.sortedHashes[j]
    })
}
```

Code: Visualizing the Execution

Measuring Efficiency

- To prove the theory, we run a simulation: 2,000 Keys on 3 Nodes, then add a 4th.
- **Standard Modulo**: Would have forced ~75% (1,500 keys) to move.
- **Consistent Hashing**: Only ~22.95% (459 keys) moved.
- **The Result**: Near-perfect rebalancing efficiency.

Below is the implementation under *simulation.go*:

```go
// Rebalancing check
movedCount := 0
for _, key := range keys {
    prevNode := keyMap[key]        // Valid before scaling
    currNode := s.Ring.GetNode(key) // Valid after scaling

    if prevNode != currNode {
        movedCount++
    }
}
fmt.Printf("Total Moved: %d keys (%.2f%%)", movedCount, float64(movedCount)/float64(len(keys))*100)
```

Code: Full Simulation Output

Visualizing the process

Below we can see the detailed output of the program, using the simulation of 2000 total keys, 3 initial nodes, followed by adding another one:

```
Distributed-Systems-Simplified\Consistent Hashing> go run .
=== Consistent Hashing Simulation ===
Total Keys: 2000

[Step 1] Initial Cluster: [Node-A Node-B Node-C]

--- Node Distribution ---
Node-A  : [                    ]  685 keys (34.2%)
Node-B  : [                    ]  620 keys (31.0%)
Node-C  : [                    ]  695 keys (34.8%)

[Step 2] Adding Node-D to the cluster...
  -> Key user-session-6 moved from Node-C to Node-D
  -> Key user-session-14 moved from Node-C to Node-D
  -> Key user-session-15 moved from Node-C to Node-D
  -> Key user-session-16 moved from Node-C to Node-D
  -> Key user-session-17 moved from Node-C to Node-D
  ... (total 459 keys moved)

--- Node Distribution ---
Node-A  : [                    ]  600 keys (30.0%)
Node-B  : [                    ]  571 keys (28.5%)
Node-C  : [                    ]  370 keys (18.5%)
Node-D  : [                    ]  459 keys (22.9%)

[Analysis]
Total Moved: 459 keys (22.95%)
Ideal Movement: 25.00% (1/4 of keys)
Result: Very efficient rebalancing! (Modulo hashing would move ~75%)
```

Summary & Takeaways

STOP THE WORLD? NO

Scale dynamically without massive data migrations or downtime.

VIRTUAL NODES

Essential to avoid data skew and "Hot Shards".

THE RING

Thinking in circular space simplifies complex "wrap-around" coordination.

INDUSTRY STANDARD

The backbone of Discord, Cassandra, DynamoDB, and modern CDNs.

Want to dive deeper?

Explore the full source code and deep-dive documentation on my GitHub repository or connect via LinkedIn.

GitHub LinkedIn

LIKE • COMMENT • SHARE